Phala Network 프론트엔드 SDK 구현 분석

1. Phala SDK 개요

Phala SDK는 **Phat Contract와 상호작용하기 위한 JavaScript/TypeScript SDK**로, 웹 애플리케이션과 Node.js 환경에서 Phala Network의 TEE 기반 스마트 컨트랙트를 쉽게 사용할 수 있게 해주는 라이브러리입니다.

2. SDK 구조 분석

2.1 패키지 정보

```
{
"name": "@phala/sdk",
"version": "0.6.0-beta.16",
"description": "Phala Phat Contract JS SDK",
// 멀티 플랫폼 지원
"main": "dist/node/index",
"browser": "dist/browser/index",
// ES Module 및 CommonJS 지원
"exports": {
".": {
"import": {
"node": "./dist/node/index.mjs",
```

```
"default": "./dist/browser/index.mjs"
},
"require": {
"node": "./dist/node/index.js",
"default": "./dist/browser/index.js"
}
}
}
}
```

2.2 핵심 의존성

```
// Polkadot 생태계 통합
├── @polkadot/api: Substrate API 클라이언트
├── @polkadot/api-contract: 스마트 컨트랙트 API
├── @polkadot/types: 타입 정의
├── @polkadot/keyring: 키 관리
├── @polkadot/util-crypto: 암호화 유틸리티

// 브라우저 호환성
├── browserify-cipher: 암호화 폴리필
├── randombytes: 랜덤 바이트 생성
├── cross-fetch: 크로스 플랫폼 fetch
```

```
// 기타 유틸리티
├── protobufjs: Protocol Buffers 지원
├── ramda: 함수형 프로그래밍 유틸리티
├── rxjs: 반응형 프로그래밍
└── viem: Ethereum 클라이언트 (EVM 호환성)
```

3. SDK 아키텍처

3.1 모듈 구조 (추정)

```
// SDK 모듈 구조
export namespace PhalaSDK {
// 핵심 클라이언트
export class PhalaSdkClient {
// 블록체인 연결
api: ApiPromise;
// 계정 관리
keyring: Keyring;
// 컨트랙트 팩토리
contractFactory: ContractFactory;
}
// 컨트랙트 관리
```

```
export class PhatContract {
// 컨트랙트 주소
address: string;
// ABI 정보
abi: ContractAbi;
// 쿼리 메서드
query: QueryMethods;
// 트랜잭션 메서드
tx: TransactionMethods;
}
// 클러스터 관리
export class ClusterClient {
// 클러스터 정보
clusterId: string;
// RPC 엔드포인트
rpcEndpoint: string;
// 워커 목록
workers: WorkerInfo[];
}
// 유틸리티
export namespace Utils {
export function encodeCall(call: any): Uint8Array;
export function decodeResult(result: Uint8Array): any;
```

```
export function signMessage(message: string, keyPair: KeyringPair): string;
}
}
```

3.2 빌드 시스템

```
// tsup 설정 (추정)
// tsup.node.ts
export default {
entry: ['src/index.ts'],
format: ['cjs', 'esm'],
target: 'node18',
platform: 'node',
outDir: 'dist/node',
dts: true,
clean: true,
external: [
'@polkadot/api',
'@polkadot/keyring',
'crypto',
'fs',
'path'
]
```

```
}

// tsup.browser.ts
export default {
entry: ['src/index.ts'],
format: ['cjs', 'esm'],
target: 'es2020',
platform: 'browser',
outDir: 'dist/browser',
dts: true,
clean: true,
define: {
global: 'globalThis',
'process.env.NODE_ENV': '"production"'
},
esbuildPlugins: [
NodeGlobalsPolyfillPlugin({
buffer: true,
process: true
}),
NodeModulesPolyfillPlugin()
]
```

```
}
```

4. 핵심 기능 구현

4.1 SDK 클라이언트 초기화

```
// SDK 클라이언트 구현 (추정)
export class PhalaSdkClient {
private api: ApiPromise;
private keyring: Keyring;
private clusters: Map<string, ClusterClient>;
constructor(config: SdkConfig) {
this.keyring = new Keyring({ type: 'sr25519' });
this.clusters = new Map();
}
async connect(endpoint: string): Promise<void> {
// 1. Substrate API 연결
const wsProvider = new WsProvider(endpoint);
this.api = await ApiPromise.create({
provider: wsProvider,
types: phalaTypes,
rpc: phalaRpc
});
```

```
// 2. 체인 정보 확인
const chain = await this.api.rpc.system.chain();
const version = await this.api.rpc.system.version();
console.log(`Connected to ${chain} v${version}`);
// 3. 이벤트 리스너 설정
this.setupEventListeners();
}
async getClusterClient(clusterId: string): Promise<ClusterClient> {
if (!this.clusters.has(clusterId)) {
// 클러스터 정보 조회
const clusterInfo = await this.api.query.phalaComputation.clusters(clusterId
if (clusterInfo.isNone) {
throw new Error(`Cluster ${clusterId} not found`);
}
// 클러스터 클라이언트 생성
const client = new ClusterClient(clusterId, clusterInfo.unwrap());
this.clusters.set(clusterId, client);
}
return this.clusters.get(clusterId)!;
}
private setupEventListeners(): void {
// 블록체인 이벤트 구독
this.api.query.system.events((events) => {
```

```
events.forEach((record) => {
const { event } = record;
if (this.api.events.phalaComputation.ContractInstantiated.is(event)) {
this.handleContractInstantiated(event);
} else if (this.api.events.phalaComputation.Transferred.is(event)) {
this.handleTransfer(event);
}
});
});
}
}
```

4.2 Phat Contract 인터페이스

```
// Phat Contract 구현 (추정)
export class PhatContract {
private clusterId: string;
private contractId: string;
private abi: ContractAbi;
private client: ClusterClient;
constructor(
clusterId: string,
contractId: string,
```

```
abi: ContractAbi,
client: ClusterClient
) {
this.clusterId = clusterId;
this.contractId = contractId;
this.abi = abi;
this.client = client;
}
// 읽기 전용 쿼리
async query<T = any>(
method: string,
args: any[] = [],
options: QueryOptions = {}
): Promise<T> {
// 1. 메서드 시그니처 생성
const selector = this.abi.getMethodSelector(method);
// 2. 인자 인코딩
const encodedArgs = this.abi.encodeArgs(method, args);
// 3. 쿼리 페이로드 생성
const payload = new Uint8Array([...selector, ...encodedArgs]);
// 4. 클러스터에 쿼리 전송
const result = await this.client.query({
contractId: this.contractId,
```

```
inputData: payload,
origin: options.origin ||  null,
gasLimit: options.gasLimit ||  null,
storageDepositLimit: options.storageDepositLimit ||  null
});
// 5. 결과 디코딩
if (result.result.isOk) {
return  this.abi.decodeOutput(method,  result.result.asOk.data);
}  else  {
throw  new  Error(`Query failed: ${result.result.asErr}`);
}
}
// 상태 변경 트랜잭션
async  tx(
method: string,
args: any[] = [],
options: TxOptions =  {}
): Promise<TxResult>  {
// 1. 트랜잭션 페이로드 생성
const  selector  =  this.abi.getMethodSelector(method);
const  encodedArgs  =  this.abi.encodeArgs(method,  args);
const  payload  =  new  Uint8Array([...selector,  ...encodedArgs]);
// 2. 트랜잭션 생성
```

```
const tx = this.client.api.tx.phalaComputation.pushMessage(
this.contractId,
payload
);
// 3. 서명 및 전송
const keyPair = options.account || this.client.defaultAccount;
const result = await tx.signAndSend(keyPair, (status) => {
if (status.isInBlock) {
console.log(`Transaction included in block ${status.asInBlock}`);
} else if (status.isFinalized) {
console.log(`Transaction finalized in block ${status.asFinalized}`);
}
});
return {
txHash: result.toString(),
blockHash: result.toString(), // 실제로는 이벤트에서 추출
events: [] // 실제로는 이벤트 파싱 결과
};
}
}
```

4.3 클러스터 클라이언트

```
// 클러스터 클라이언트 구현 (추정)
export class ClusterClient {
private clusterId: string;
private rpcEndpoint: string;
private workers: WorkerInfo[];
private httpClient: HttpClient;
constructor(clusterId: string, clusterInfo: any) {
this.clusterId = clusterId;
this.rpcEndpoint = this.selectRpcEndpoint(clusterInfo.workers);
this.workers = clusterInfo.workers;
this.httpClient = new HttpClient(this.rpcEndpoint);
}
async query(request: QueryRequest): Promise<QueryResponse> {
// 1. 요청 직렬화
const serializedRequest = this.serializeQueryRequest(request);
// 2. HTTP 요청 전송
const response = await this.httpClient.post('/query', {
body: serializedRequest,
headers: {
'Content-Type': 'application/octet-stream'
}
});
```

```
// 3. 응답 역직렬화
return this.deserializeQueryResponse(response);
}
async getContractInfo(contractId: string): Promise<ContractInfo> {
const response = await this.httpClient.get(`/contracts/${contractId}`);
return response.json();
}
async getClusterInfo(): Promise<ClusterInfo> {
const response = await this.httpClient.get('/info');
return response.json();
}
private selectRpcEndpoint(workers: WorkerInfo[]): string {
// 워커 중에서 가장 신뢰할 수 있는 엔드포인트 선택
const activeWorkers = workers.filter(w => w.state === 'Ready');
if (activeWorkers.length === 0) {
throw new Error('No active workers found in cluster');
}
// 라운드 로빈 또는 랜덤 선택
const selectedWorker = activeWorkers[Math.floor(Math.random() * activeWorker
return selectedWorker.rpcEndpoint;
}
}
```

5. 타입 정의 및 인터페이스

5.1 핵심 타입 정의

```
// 타입 정의 (추정)
export  interface SdkConfig {
// 네트워크 설정
endpoint?: string;
// 기본 계정
defaultAccount?: KeyringPair;
// 타임아웃 설정
timeout?: number;
// 로깅 레벨
logLevel?: 'debug' | 'info' | 'warn' | 'error';
}

export  interface ContractAbi {
// 컨트랙트 메타데이터
metadata: ContractMetadata;
// 메서드 정의
methods: MethodDefinition[];
// 이벤트 정의
events: EventDefinition[];
```

```
// 유틸리티 메서드
getMethodSelector(method: string): Uint8Array;
encodeArgs(method: string, args: any[]): Uint8Array;
decodeOutput(method: string, data: Uint8Array): any;
}

export  interface QueryOptions {
// 호출자 계정
origin?: string;
// 가스 제한
gasLimit?: number;
// 스토리지 보증금 제한
storageDepositLimit?: number;
// 타임아웃
timeout?: number;
}

export  interface TxOptions extends QueryOptions {
// 서명 계정
account?: KeyringPair;
// 논스
nonce?: number;
// 팁
```

```
tip?: number;
}

export  interface WorkerInfo {
// 워커 공개키
pubkey: string;
// 워커 상태
state: 'Ready' | 'Working' | 'Paused';
// RPC 엔드포인트
rpcEndpoint: string;
// 신뢰도 점수
confidenceLevel: number;
}

export  interface ClusterInfo {
// 클러스터 ID
id: string;
// 소유자
owner: string;
// 워커 목록
workers: WorkerInfo[];
// 상태
state: 'Active' | 'Inactive';
```

```
}
```

5.2 Protocol Buffers 통합

```
// Protocol Buffers 메시지 정의 (추정)
export namespace Proto {
export interface QueryRequest {
contractId: string;
inputData: Uint8Array;
origin?: string;
gasLimit?: number;
storageDepositLimit?: number;
}
export interface QueryResponse {
result: {
ok?: {
data: Uint8Array;
gasConsumed: number;
storageDeposit: number;
};
err?: string;
};
}
```

```
export interface ContractCallRequest {
contractId: string;
selector: Uint8Array;
input: Uint8Array;
transferredValue: number;
gasLimit: number;
storageDepositLimit?: number;
}
}

// Protocol Buffers 직렬화/역직렬화
export class ProtoCodec {
static encodeQueryRequest(request: Proto.QueryRequest): Uint8Array {
// protobufjs를 사용한 인코딩
const message = QueryRequestProto.create(request);
return QueryRequestProto.encode(message).finish();
}
static decodeQueryResponse(data: Uint8Array): Proto.QueryResponse {
// protobufjs를 사용한 디코딩
return QueryResponseProto.decode(data);
}
}
```

6. 유틸리티 함수

6.1 암호화 유틸리티

```
// 암호화 유틸리티 (추정)
export namespace CryptoUtils {
export function generateKeyPair(type: 'sr25519' | 'ed25519' = 'sr25519'): Keyr
const keyring = new Keyring({ type });
return keyring.addFromUri(mnemonicGenerate());
}
export function signMessage(message: string, keyPair: KeyringPair): string {
const signature = keyPair.sign(stringToU8a(message));
return u8aToHex(signature);
}
export function verifySignature(
message: string,
signature: string,
publicKey: string
): boolean {
try {
const messageU8a = stringToU8a(message);
const signatureU8a = hexToU8a(signature);
const publicKeyU8a = hexToU8a(publicKey);
```

```
return signatureVerify(messageU8a, signatureU8a, publicKeyU8a).isValid;
} catch {
return false;
}
}
export function encryptData(data: Uint8Array, publicKey: string): Uint8Array {
// NaCl 또는 다른 암호화 라이브러리 사용
// 실제 구현은 사용하는 암호화 방식에 따라 달라짐
throw new Error('Not implemented');
}
}
```

6.2 네트워크 유틸리티

```
// 네트워크 유틸리티 (추정)
export class HttpClient {
private baseUrl: string;
private timeout: number;
constructor(baseUrl: string, timeout: number = 30000) {
this.baseUrl = baseUrl;
this.timeout = timeout;
}
async get(path: string, options: RequestOptions = {}): Promise<Response> {
```

```
return this.request('GET', path, options);
}
async post(path: string, options: RequestOptions = {}): Promise<Response> {
return this.request('POST', path, options);
}
private async request(
method: string,
path: string,
options: RequestOptions
): Promise<Response> {
const url = `${this.baseUrl}${path}`;
const controller = new AbortController();
// 타임아웃 설정
const timeoutId = setTimeout(() => controller.abort(), this.timeout);
try {
const response = await fetch(url, {
method,
headers: {
'Content-Type': 'application/json',
...options.headers
},
body: options.body,
signal: controller.signal
```

```
});
if (!response.ok) {
throw new Error(`HTTP ${response.status}: ${response.statusText}`);
}
return response;
} finally {
clearTimeout(timeoutId);
}
}
}
```

7. 테스트 및 개발 도구

7.1 테스트 설정

```
// Vitest 테스트 설정 (추정)
import { describe, it, expect, beforeAll, afterAll } from 'vitest';
import { PhalaSdkClient, PhatContract } from '../src';

describe('Phala SDK', () => {
let client: PhalaSdkClient;
let contract: PhatContract;
beforeAll(async () => {
```

```
// 테스트 환경 설정
client = new PhalaSdkClient({
endpoint: 'ws://localhost:9944',
logLevel: 'debug'
});
await client.connect();
});
afterAll(async () => {
// 정리 작업
await client.disconnect();
});
it('should connect to blockchain', async () => {
expect(client.isConnected()).toBe(true);
});
it('should query contract', async () => {
const result = await contract.query('get_value');
expect(result).toBeDefined();
});
it('should send transaction', async () => {
const result = await contract.tx('set_value', [42]);
expect(result.txHash).toBeDefined();
});
```

```
});
```

7.2 개발 도구

```
// 개발 스크립트
{
"scripts": {
// 빌드
"build": "concurrently 'yarn:build:*'",
"build:node": "tsup --config tsup.node.ts",
"build:browser": "tsup --config tsup.browser.ts",
// Protocol Buffers 생성
"generate:proto": "scripts/build_proto.sh",
// 테스트
"test": "vitest --ui --coverage",
"test:ci": "vitest --run",
// 코드 품질
"prettier": "prettier --write '**/*.{ts,js,json,md}'",
"eslint": "eslint --fix '**/*.{ts,js}'"
}
}
```

이 문서는 Phala Network의 프론트엔드 SDK 구현을 다룹니다. 다음 문서에서는 E2E 테스트 시스템을 분석합니다.